



Guide d'étude

APP2 SNTETM – Session S1

Introduction à la programmation en robotique

Génie robotique
Faculté de génie
Université de Sherbrooke

Automne 2026

Note : En vue d'alléger le texte, le masculin est utilisé pour désigner les femmes et les hommes.

Document GRO120-A26-Guide_études.docx
Rédigé par François Ferland et Eugène Morin (2026).

Copyright © 2026, Faculté de génie, Université de Sherbrooke

Table des matières

1.	Activités pédagogiques et compétences.....	1
2.	Synthèse de l'évaluation	1
3.	Qualités de l'ingénieur.....	2
4.	Énoncé de la problématique.....	3
5.	Connaissances nouvelles.....	6
6.	Guide de lecture	7
6.1.	Séquence d'étude suggérée	7
7.	Environnement logiciel.....	8
8.	Sommaire des activités liées à l'unité	8
9.	Productions à remettre	9
9.1.	Formation des équipes	9
9.2.	Contraintes et pénalités pour les livrables	9
9.3.	Livrables	9
10.	Évaluations.....	11
10.1.	Rapport d'APP et validation	11
10.2.	Évaluation sommative.....	11
10.3.	Évaluation finale	11
10.4.	Grille d'évaluation du rapport et de la validation	12
11.	Activités procédurale 1	13
11.1.	Concepts généraux	13
11.2.	Langage Python	13
12.	Activité en laboratoire 1.....	16
12.1.	Opérateurs mathématiques et fonctions	16
12.2.	Les tableaux	17
13.	Activités procédurale 2	19
13.1.	Variables et données mutables.....	19
13.2.	Opérations arithmétiques et booléennes	20
13.3.	Diagramme d'activité.....	22
14.	Activité en laboratoire 2.....	24
15.	Activités procédurale 3	26
15.1.	Types et manipulation de listes.....	26
15.2.	Entrées et sorties	27
15.3.	Définition de fonctions	28
15.4.	Définition de module	29
16.	Validation pratique de la solution à la problématique	30
17.	Deuxième tutorat.....	31
18.	Comparaison avec le C++	32
18.1.	Importation.....	37
18.2.	Déclaration de variables, énoncés et commentaires	37
18.3.	Blocs et portée lexicale	39
18.4.	Déclaration de fonctions	40
18.5.	Opérateurs booléens et <i>switch-case</i>	41

1. Activités pédagogiques et compétences

GRO120 : Introduction à la programmation en robotique

1. Faire la synthèse et l'implémentation d'un algorithme en vue de résoudre un problème selon une approche procédurale.
2. Mettre en œuvre un programme informatique selon les meilleures pratiques d'organisation du code, de documentation et de validation de son fonctionnement.

2. Synthèse de l'évaluation

La note attribuée aux activités pédagogiques de l'APP est une note individuelle, sauf pour le rapport d'APP qui est une note par équipe de deux. L'évaluation porte sur les compétences figurant dans la description des activités pédagogiques de l'APP à la section 1. Ces compétences, ainsi que la pondération (sur un total de 4500 points pour une session de 15 crédits) de chacune d'entre elles dans l'évaluation de l'unité, sont :

<i>Activité pédagogique</i>	GRO120		
<i>Compétence</i>	IaG ¹	C1 Algorithmes	C2 Programmes
<i>Livrables</i>			
Rapport d'APP	🟡 Niveau 1	30	30
Validation	🔴 Niveau 0	15	15
Évaluation sommative théorique	🔴 Niveau 0	135	135
Évaluation finale théorique	🔴 Niveau 0	120	120
Total		300	300

¹ Utilisation de l'intelligence artificielle générative

Pour connaître les spécificités liées aux balises d'utilisation des outils d'intelligence artificielle générative (IaG) autorisée, référez-vous au tableau suivant.

Lien : [Balises d'utilisation des outils d'IaG - Service de soutien à la formation - Université de Sherbrooke](#)

NIVEAU 0 : L'utilisation de l'IaG lors de l'évaluation sommative et de l'évaluation finale **est interdite**.

*Ceci signifie que si la personne enseignante a un motif de croire qu'il y a eu l'utilisation d'une IaG dans une situation d'évaluation, elle doit dénoncer les faits auprès de la personne responsable des dossiers disciplinaires universitaires. Il s'agit d'un délit relatif aux études tel que stipulé dans le Règlement des études.

NIVEAU 1 : L'utilisation de l'IAg pour la résolution de la problématique, la préparation à la validation ainsi que pour tout autre livrable en lien avec la résolution de la problématique, **est autorisée uniquement pour assister l'apprentissage et la réalisation de vos travaux**. Les outils pourraient être bénéfiques pour vous aider à structurer votre pensée et pour produire des textes de qualité. Lorsque vous utilisez les outils d'IAg, vous êtes tenus de **déclarer les usages que vous en avez faits** en utilisant le formulaire de déclaration fourni sur le site Moodle de la session.

3. Qualités de l'ingénieur

Les qualités de l'ingénieur visées par cette unité d'APP sont les suivantes. D'autres qualités peuvent être présentes sans être visées ou évaluées dans cette unité d'APP.

	Q01	Q02	Q03	Q04	Q05	Q06	Q07	Q08	Q09	Q10	Q11	Q12
Touchée	x	x	x		x							
Évaluée	x	x	x		x							

Les qualités de l'ingénieur sont les suivantes. Pour une description détaillée des qualités et leur provenance, consultez le lien suivant :

<https://www.usherbrooke.ca/genie/futurs-etudiants/programmes-de-1er-cycle/baccalaureats/bcapg>

Qualité	Libellé
Q01	Connaissances en génie
Q02	Analyse de problèmes
Q03	Investigation
Q04	Conception
Q05	Utilisation d'outils d'ingénierie
Q06	Travail individuel et en équipe
Q07	Communication
Q08	Professionnalisme
Q09	Impact du génie sur la société et l'environnement
Q10	Déontologie et équité
Q11	Économie et gestion de projets
Q12	Apprentissage continu

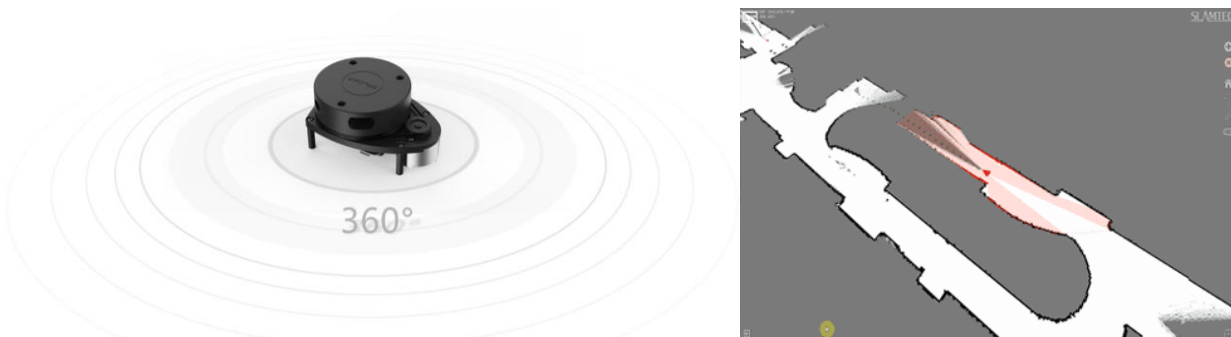
4. Énoncé de la problématique

Correction de capteurs limités (ou compenser le matériel par le logiciel)

Cet automne, vous êtes stagiaires dans une entreprise produisant des aspirateurs robotisés capable de parcourir de façon autonome des appartements de taille moyenne et récupérer l'entière de la poussière en quelques heures seulement. Cet aspirateur, apparu sur le marché il y a déjà quelques années, a eu un succès instantané malgré (ou grâce à) la simplicité de l'appareil. En effet, le système de guidage dépend entièrement d'une paire de capteurs infrarouges donnant la distance relative des obstacles dans des cônes d'une trentaine de degrés d'ouverture et d'un interrupteur monté sur un pare-chocs permettant de détecter lorsque l'aspirateur frappe un mur. Les déplacements de l'aspirateur sont majoritairement aléatoires, avec quelques ajustements pour éviter les obstacles détectés ou pour changer de cap lorsqu'un obstacle a été rencontré.

Or, la compétition est de plus en plus féroce. Bien que l'entreprise jouît d'une forte reconnaissance sur le marché, le nom du produit étant pratiquement un synonyme d'« aspirateur robot », les ventes s'essouffent petit à petit. En effet, des produits beaucoup plus sophistiqués, capable notamment de cartographier l'environnement pour effectuer une planification plus efficace de leurs déplacements, sont maintenant disponibles sur le marché, et ce à prix compétitif. La haute direction n'est pas dupe, et elle a lancé un vaste projet pour moderniser l'ensemble de la technologie intégrée à ses appareils.

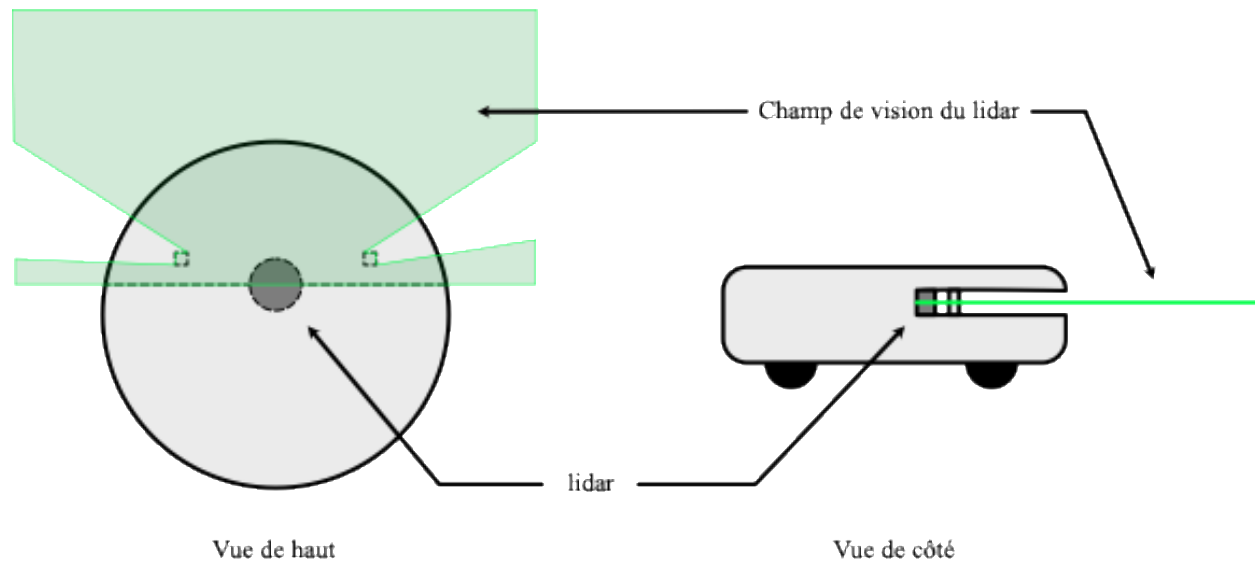
C'est dans ce contexte que vous avez été recrutés, plus spécifiquement dans le groupe « Perception et Planification ». Votre superviseur explique le rôle de l'équipe : moderniser les capteurs à bord de l'aspirateur pour une perception plus juste de l'environnement. La solution pour l'instant repose entièrement sur l'ajout d'un lidar : un capteur capable de percevoir les obstacles sur 360 degrés à l'aide d'un laser rotatif.



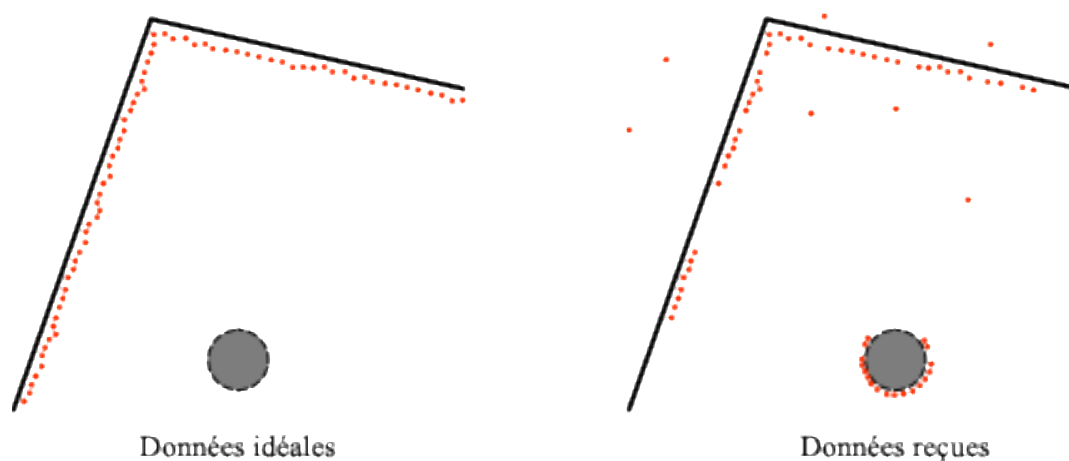
Exemple de lidar 2D à 360 degrés et capture en direct (en rouge) et accumulée (en tons de gris) –
www.slamtech.com

Bien que la technologie lidar ne soit pas nouvelle, elle est coûteuse à intégrer, surtout pour des robots domestiques. Or, la miniaturisation des composantes optiques nécessaires et la simplification de l'électronique embarquée rend l'option intéressante pour l'entreprise qui vous embauche.

Il reste cependant des défis d'intégration importants. La division « Châssis et Capteurs » de l'entreprise travaille depuis un certain temps à la conception d'un capteur intégré à la coque extérieure du robot et est arrivé à ce concept :



Comme on peut le voir, le champ de vision du lidar (en vert) est limité à 180 degrés et obstrué par les supports du châssis. De plus, par soucis d'économie, les composants optiques et électroniques choisies donnent un signal très bruité. Voici un exemple comparant les données idéales et les données brutes reçues du prototype lorsque le robot est près d'un coin de mur :



On remarque non seulement les obstructions du châssis, mais aussi quelques données dans les lectures. Il est trop tard dans le processus de conception pour revenir sur les choix matériels de l'entreprise, mais votre superviseur est confiant que le logiciel embarqué du robot sera en mesure de compenser pour ces erreurs.

Avant de développer ce logiciel embarqué, votre superviseur souhaite d'abord mettre en place une structure de validation afin de raffiner le fonctionnement du système de correction sans avoir besoin d'accéder au prototype physique du robot. Cette structure sera en fait un logiciel Python

capable de charger depuis des fichiers texte des données représentatives provenant du lidar et appliquer différents filtres correctifs pour ensuite produire de nouveaux fichiers textes des données corrigées.

Pour le filtrage, il s'attend à ce que vous implémentiez au moins trois algorithmes :

- Une limitation par plage minimum-maximum, où les données sortant de ces limites seront soit plafonnées (pour les valeurs au-delà du maximum), soit marquées comme inconnues (pour les données sous le minimum). Les valeurs inconnues seront indiquées par des valeurs numériques inférieures à zéro (par exemple -1.0).
- Un filtrage par moyenne mobile, où une fenêtre de 3 valeurs seront utilisées pour calculer une moyenne et produire une nouvelle valeur. Par exemple, pour la séquence (1, 3, 2, 4, 5, 3), on obtiendrait (2.0, 2.0, 3.0, 3.7, 4.0, 4.0). Aux extrémités, on utilise que deux valeurs (donc 1 et 3 pour la première sortie et 5 et 3 pour la dernière).
- Un filtrage par médiane glissante. Sensiblement la même approche que la moyenne glissante avec une fenêtre de 3 valeurs, mais en utilisant la médiane de la distribution. Ainsi, les valeurs de sorties pour la même séquence d'entrée seraient (1, 2, 3, 4, 4, 3). Aux extrémités, on reprend la valeur initiale plutôt que tenter de trouver la médiane d'une distribution à deux valeurs.

Le programme final devra donc pouvoir utiliser n'importe quel fichier par argument à la ligne de commande, permettre d'indiquer le nom du fichier de sortie, puis demander à l'utilisateur de choisir le filtre à appliquer (1, 2 ou 3). Le programme devra afficher ces instructions à l'utilisateur. De plus, le format des données de sorties devrait être le même que pour les données d'entrées afin de pouvoir enchaîner plusieurs traitements sans modifier les fichiers intermédiaires.

Finalement, pour des fins de statistiques, le programme devra afficher les valeurs minimales, maximales, médianes et moyennes basées sur l'ensemble des données valides (donc plus grandes ou égales à zéro) étant produites après filtrage (de sortie). Un affichage du nombre de points d'entrées et du nombre de points valides est également demandé.

Votre superviseur souhaite une approche modulaire où les différentes responsabilités du programme seront bien délimitées en fonctions et modules Python afin qu'elles puissent être réintégrées dans d'autres logiciels de l'entreprise. De plus, il souhaite que chaque fonction implémentant un algorithme soit accompagnée d'un diagramme d'activité décrivant son fonctionnement, et que l'ensemble du code (fonctions et modules) soit documenté selon les règles de l'art avec des *Docstrings* Python. Finalement, vous devrez produire une série de tests unitaires automatisés qui valideront le bon fonctionnement des fonctions implémentées dans le module de filtrage. Votre superviseur vous demande également d'appliquer de bonnes pratiques de programmation, notamment en séparant clairement les responsabilités de vos fonctions et modules, en limitant les variables globales et en utilisant des identifiants clairs et intuitifs pour le futur lecteur de votre code.

Des exemples de données d'entrées et de sorties vous sont fournies avec un squelette de code à compléter et remettre à votre superviseur d'ici environ 2 semaines.

5. Connaissances nouvelles

Connaissances déclaratives : *Quoi*

- Programmation procédurale (GRO120-1) :
 - Variables : déclaration, initialisation et assignation ;
 - Portée lexicale ;
 - Évaluation séquentielle et instantanée ;
 - Données mutables ;
 - Expressions arithmétiques et logiques ;
 - Structures de données de base : tableaux fixes, vecteurs et types agrégés ;
 - Structures de contrôle de flux.
- Éléments fondamentaux d'un programme informatique (GRO120-2) :
 - Entrées et sorties (fichiers texte et flux interactifs)
 - Fonctions et modules
- Gestion automatique de la mémoire (GRO120-2)
- Validation automatisée (GRO120-2)
- Environnement de développement intégré : édition, exécution et débogage (GRO120-2)
- Documentation d'un programme : commentaires et diagrammes d'activité (GRO120-2)

Connaissances procédurales : *Comment*

- Implémenter un algorithme à l'aide d'un langage de programmation procédurale
- Utiliser des diagrammes d'activités UML pour développer des algorithmes
- Organiser l'information à l'aide de structures de données de base
- Tester et valider un algorithme de façon automatisée
- Construire des programmes en utilisant une approche modulaire à l'aide de fonctions
- Utiliser un environnement de développement intégré pour programmer, tester et déboguer un logiciel
- Exploiter les services d'entrées et sorties de base d'un système d'exploitation
- Documenter le fonctionnement d'un programme

Connaissances conditionnelles : *Quand*

- Sélectionner la structure de données appropriée pour représenter l'état d'un algorithme
- Choisir une bonne séparation de responsabilités pour organiser un programme.

6. Guide de lecture

Volume obligatoire :

- Swinnen, Gérard. « Apprendre à programmer avec Python 3 ». Disponible librement en ligne : <https://inforef.be/swi/python.htm>

6.1. Séquence d'étude suggérée

- **Avant le procédurale 1 :**
 - Chapitre 2 – Premiers pas (pp 11-20)
 - Chapitre 3 – Contrôle du flux d'exécution (pp. 21-26)
 - Chapitre 4 – Instructions répétitives (pp. 27-36)
- **Avant le laboratoire 1 :**
 - Chapitre 5 – Principaux types de données (pp. 37-48)
 - Chapitre 6 – Fonctions prédéfinies (*La fonction print()* – pp. 49)
 - Chapitre 7 – Fonctions originales (jusqu'à *Variables locales, variables globales*, pp. 61-66)
- **Avant le procédurale 2 :**
 - Chapitre 6 – Fonctions prédéfinies (*Interaction avec l'utilisateur : la fonction input()* – pp. 50-52)
 - Guide sur les diagrammes d'activités (voir site de l'APP)
- **Avant le laboratoire 2 :**
 - Présentation sur la documentation et la lisibilité (voir site de l'APP)
- **Avant le procédurale 3 :**
 - Chapitre 7 – Fonctions originales (jusqu'à *Typage des paramètres*, pp. 66-75)
 - Chapitre 9 – Manipulation des fichiers (jusqu'à *Gestion des exceptions*, pp. 107-116)
 - Chapitre 10 – Approfondir les types de données (pp. 121-125, jusqu'à *Appartenance à une séquence*)
- **Après la seconde activité procédurale :**
 - Section 18 du guide d'étude : « Comparaison avec le C++ »

7. Environnement logiciel

L'ensemble du cours utilisera le langage Python 3 dans l'environnement Visual Studio Code en visant les versions disponibles sur les ordinateurs de la faculté (soit Python 3.14 et Visual Studio Code 1.139, autant sous Linux que Windows). S'il est tout à fait possible d'installer ces logiciels sur vos ordinateurs personnels, et il existe beaucoup de ressources en ligne à cet effet pour vous aider, **sachez que l'équipe d'enseignement de l'APP n'est pas tenue de vous assister dans la configuration de votre équipement personnel**. Si vous éprouvez des difficultés à installer ces logiciels, on vous suggère fortement d'effectuer vos exercices et la résolution de l'APP sur les postes de la faculté.

8. Sommaire des activités liées à l'unité

Semaine 1 (28 septembre au 2 octobre) :

- 1^{re} rencontre de tutorat
- Étude personnelle et exercices
- Formation à la pratique procédurale 1
- Formation en laboratoire 1
- Formation à la pratique procédurale 2
- Formation en laboratoire 2
- Formation à la pratique procédurale 3

Semaine 2 (5 octobre au 9 octobre) :

- Soutien en laboratoire pour la résolution de la problématique
- Validation de la problématique
- Remise du rapport d'APP **par équipe de deux**.
- 2^e rencontre de tutorat
- Évaluation formative
- Séminaire de Python vers le C++
- Consultation pré-examen
- Évaluation sommative théorique

9. Productions à remettre

Vous démontrerez le fonctionnement de votre solution lors de la validation pratique en laboratoire (mardi 6 octobre de 13h00 à 16h00). Ensuite, vous devrez rédiger un rapport d'APP (à remettre mercredi le 7 octobre avant 8h30). Les consignes de rédaction du rapport de l'unité d'APP sont données à la section 9.3. Le code de votre solution devra être remis au même moment que le rapport. Les instructions pour le dépôt seront diffusées sur le site de l'APP.

9.1. Formation des équipes

La résolution de la problématique se fera en **équipe de deux**. La problématique peut être résolue entièrement par ordinateur, nous nous attendons à ce que tous les membres de l'équipe soient en mesure d'en faire la démonstration. La façon d'inscrire les équipes sera diffusée sur le site de l'APP. S'il est impossible pour vous de rejoindre une équipe, par exemple car le nombre d'étudiants dans le groupe est impair, veuillez en avvertir le tuteur le plus vite possible.

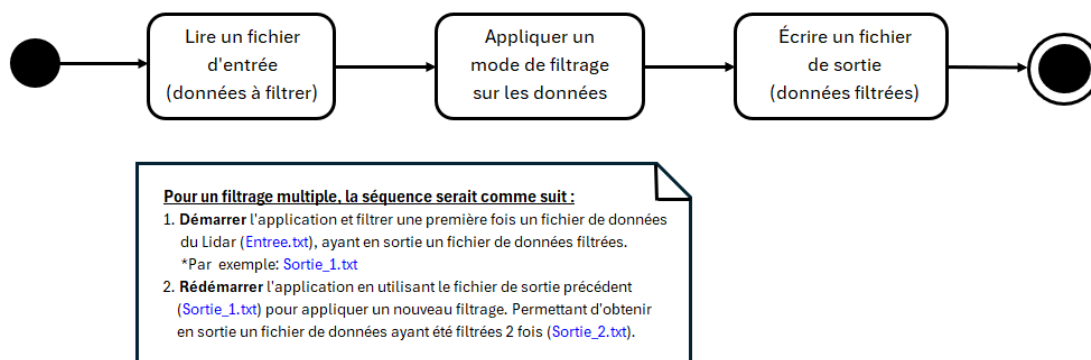
9.2. Contraintes et pénalités pour les livrables

Veuillez identifier avec votre nom, prénom, numéro de matricule et CIP tous les documents que vous remettrez. Tout retard sur la remise d'un livrable entraîne **une pénalité de 20 % par jour** sur la note associée au rapport de l'APP.

9.3. Livrables

Code

Pour nous permettre d'évaluer votre solution, vous devrez remettre la totalité du code Python que vous aurez écrit. Celui-ci devra pouvoir s'exécuter sans modifications de l'environnement décrit à la section 7. Voici ci-après un diagramme simplifié, du processus de traitement attendu lors de l'exécution du module **banc_lidar.py** (voir aussi les commentaires dans ce module du livrable).



Le code produit devra être correctement documenté et pourra être exécuté de la façon décrite dans l'énoncé de la problématique. En d'autres termes, la personne qui corrigera votre travail ne devrait pas avoir à modifier quoique ce soit dans votre code pour évaluer son bon fonctionnement.

Rapport

Dans un document de **maximum 5 pages** (sans compter la page titre et la table des matières), vous devrez décrire la façon dont vous avez résolu le problème. Voici les informations recherchées :

Identification du problème. Décrivez clairement le problème à résoudre. Ici, il ne faut pas répéter le texte de l'énoncé de la problématique, mais décrire plus spécifiquement quel était le problème à résoudre à partir de votre analyse des données d'entrées fournies pour l'APP. Par exemple, quelles étaient les valeurs minimales et maximales définissant la plage des données valides du lidar ?

Procédure de résolution. Décrivez comment vous avez élaboré votre solution pour les algorithmes de traitement des données. **Fournissez des diagrammes d'activités de chacun des algorithmes implémentés.** On ne veut pas que vous décriviez chaque ligne de code, mais plutôt votre raisonnement dans la construction de vos algorithmes.

Mise en œuvre du programme. Cet aspect sera surtout évalué par le code que vous avez produit, mais décrivez ici brièvement la façon dont est organisé votre code, comment vous avez regroupé certaines fonctions en modules en quoi les tests développés sont suffisants pour valider le bon fonctionnement de vos algorithmes.

Analyse du résultat. Présentez une analyse critique du fonctionnement de votre programme. Quelles sont ses forces ? Quels sont ses limitations et les points à améliorer ? Par exemple, croyez-vous que votre programme puisse être intégré tel quel dans le système embarqué du robot de l'entreprise décrite dans l'énoncé de la problématique.

10. Évaluations

Cette échelle sera utilisée pour convertir les notes à des cotes (seuil A+ à 85%) :

W ou E	D	D+	C-	C	C+	B-	B	B+	A-	A	A+
<50%	50,0%	53,0%	57,0%	60,0%	64,0%	68,0%	71,0%	75,0%	78,0%	81,0%	85,0%
0,0	1,0	1,3	1,7	2,0	2,3	2,7	3,0	3,3	3,7	4,0	4,3

10.1. Rapport d'APP et validation

La grille suivante décrit la répartition des points pour chaque élément du rapport, du code remis et de la validation :

<i>Activité pédagogique</i> <i>Compétence</i>	GRO120	
	C1	C2
<i>Contenu du rapport</i>		
1. Identification du problème (Q02.1)	10	
2. Procédure de résolution (Q02.2)	10	5
3a. Mise en œuvre du code (Q02.3)	5	10
3b. Qualité de la documentation (Q02.3)		10
4. Analyse du résultat (Q02.4)	5	5
Total du rapport	30	30
Validation de la solution	15	15

C1 : Algorithmes, C2 : Programmes

Les éléments du rapport et de la validation sont évalués selon les indicateurs des qualités de l'ingénieur qui sont détaillés à la section 10.4. **Une version en plus grand format de cette grille est également disponible sur le site de l'APP.**

10.2. Évaluation sommative

L'évaluation sommative porte sur tous les objectifs d'apprentissage de l'unité. C'est un examen théorique où **aucune documentation sera permise.**

10.3. Évaluation finale

L'évaluation finale portera également sur tous les objectifs d'apprentissage de l'activité pédagogique GRO120. L'examen aura la même forme que celle de l'évaluation sommative théorique et **aucune documentation sera permise.**

10.4. Grille d'évaluation du rapport et de la validation

Une version PDF est également disponible sur le site de l'APP.

SGRO120 - Grille d'évaluation, rapport et validation, automne 2026

[illegible]

11. Activités procédurale 1

Les objectifs de ce procédural sont :

- Comprendre certains concepts généraux de la programmation.
- Comprendre certains termes et concepts de programmation et leur application dans le langage Python.

Suggestions :

- Pour cette activité, il est avantageux d'avoir accès au volume « Apprendre à programmer avec Python 3 » et d'effectuer les lectures recommandées.

11.1. Concepts généraux

Les variables :

- Qu'est-ce qu'une variable ?
- Quelle est son utilité ?

Les tableaux :

- Qu'est-ce qu'un tableau de données ?
- Quel est son utilité ?
- Que représente une position (ou un index) dans un tableau ?

11.2. Langage Python

Les variables :

- Comment peut-on définir une variable en Python ?
- Comment peut-on attribuer une valeur à une variable ?
- Est-ce qu'une variable peut être réutilisée ?

Les tableaux :

- Comment peut-on définir un tableau en Python (donnez un exemple) ?
- Quels types d'éléments un tableau peut-il contenir ?
- Quel est l'opérateur en Python permettant d'obtenir le nombre d'éléments dans un tableau ?
- Quelle serait la valeur de la position, pour lire le PREMIER élément d'un tableau ?
- Quelle serait la valeur de la position, pour lire le DERNIER élément d'un tableau ?
- Comment pourrait-on afficher la 3e valeur (21) du tableau suivant [1, 7, 21, 19] ?
- En réutilisant le tableau de la question précédente, qu'arrive-t-il si on tente de lire l'élément à la position 4 ?
- Si un tableau est vide, qu'arrive-t-il si on tente de lire le premier élément ?
- Comment pourrait-on gérer ce contexte ?

Les conditionnels :

- Que représente les trois (3) conditionnels suivant :
 - if
 - else
 - elif
- Quels sont leurs utilités ?
- Donnez un exemple d'utilisation de conditionnel :

Les boucles :

- Qu'est-ce qu'une itération sur un tableau de données ?
- Comment peut-on itérer un tableau de données en Python ?
- Donnez un exemple :

Les fonctions :

- Qu'est-ce qu'une fonction ?
- Comment peut-on définir une fonction en Python ?
- Qu'est-ce qu'un paramètre de fonction ?
- Qu'est-ce qu'un retour de fonction ?
- Est-ce qu'une fonction retourne toujours une valeur ?

L'affichage :

- Quelle est l'instruction pour afficher un texte en console en Python ?
- Comment peut-on afficher un texte ?
- Comment peut-on afficher la valeur d'une variable ?

12. Activité en laboratoire 1

Les objectifs de ce laboratoire sont :

- Apprendre à utiliser l'environnement de développement Visual Studio Code.
- Développer des modules dans le langage Python.
- Connaitre et utiliser la programmation modulaire (fonctions).
- Connaitre et utiliser les tableaux.

Suggestions :

- Pour cette activité, il est recommandé d'avoir fait vos lectures sur le langage Python.
- Sur les ordinateurs au laboratoire, il est recommandé de travailler sur le *thawspace* (disque K:) et d'enregistrer vos exercices du laboratoire sur une clé USB, votre One Drive ou SharePoint.

12.1. Opérateurs mathématiques et fonctions

Ce second exercice cible la réalisation d'un module en Python servant aux calculs mathématiques. **NE PAS** utiliser la librairie « *math* » de Python pour cet exercice. *Vous devez donc coder par vous-même le contenu des fonctions énumérées ci-après !*

- Ouvrir l'environnement de développement *Visual Studio Code*
- Créer un module en python nommé : *mathematique.py*
- Chaque opération doit se faire dans une fonction (**def**) distincte, et doit comprendre un nom, ainsi que le ou les paramètres requis.
- Dans ce module, les opérateurs attendus sont les suivants (une valeur de retour est attendue pour chaque opérateur) :
 - **additionner** : Somme de 2 nombres entiers
 - **soustraire** : Soustraction de 2 nombres entiers
 - **multiplier** : Multiplication de 2 nombres entiers
 - **diviser** : Division de 2 nombres entiers (numérateur, dénominateur)
 - Attention, qu'arrive-t-il si l'on effectue une division par 0 ?
 - Que pourrions-nous faire pour gérer ce type de division ?
 - **min** : Trouver la valeur minimum entre 2 nombres entiers
 - **max** : Trouver la valeur maximum entre 2 nombres entiers
 - **abs** : Retourner la valeur absolue d'un nombre entier

- **moyenne1** : Calculer la moyenne de 2 nombres entiers
- **moyenne2** : Calculer la moyenne en utilisant deux des méthodes implémentées précédemment
- **exponentiel** : Selon deux nombres entiers nommés « base » et « exposant »
 - Implémenter cette méthode à l'aide d'une boucle « while »
 - Si l'exposant égal 0, quel est le retour attendu...
- Exercice supplémentaire :
 - Créer un module nommé « [testmathematique.py](#) »
 - Ensuite, importer le module « [mathematique.py](#) » dans ce nouveau module
 - Puis tester l'appel de fonctions du module « [mathematique](#) » dans le module « [testmathematique](#) »

12.2. Les tableaux

Ce troisième exercice cible la réalisation d'un module en Python permettant d'effectuer des manipulations sur des tableaux. **NE PAS** utiliser les méthodes prédéfinies du langage Python pour cet exercice.

**Vous devez coder par vous-même le contenu des méthodes énumérées ci-après !*

- Créer un module en python nommé : [tableau.py](#)
- Vous pouvez utiliser le tableau suivant pour vos tests de méthode :
[mon_tableau = \[12, 7, 8, 3, 12, 3, 1, 8\]](#)
- Dans ce module, les manipulations attendus sont les suivantes (une valeur de retour est attendue pour chaque fonction) :
 - **nombre_present** : Vérifie si un nombre est présent dans un tableau
 - Paramètres en entrés : Un tableau d'entiers, Un nombre entier
 - Retour : Booléen (retourne vrai ou faux)
 - Test possible : Rechercher le nombre « 3 » dans « [mon_tableau](#) »
 - Test possible : Rechercher le nombre « 5 » dans « [mon_tableau](#) »
 - **rechercher_nombre** : Retourne la première position détectée d'un nombre dans un tableau (retourne -1 si non présente)
 - Paramètres en entrés : Un tableau d'entiers, Un nombre entier
 - Retour : Entier (-1 si non trouvé)
 - Test possible : Rechercher le nombre « 3 » dans « [mon_tableau](#) »
 - Test possible : Rechercher le nombre « 5 » dans « [mon_tableau](#) »

- **moyenne** : Retourne la moyenne des nombres d'un tableau
 - Paramètre en entrée : Un tableau d'entiers
 - Retour : Un nombre flottant
 - Test possible : Avec « [mon_tableau](#) »
- **inverser_tableau** : Retourne un tableau dont les valeurs sont inversées.
 - Paramètre en entrée : Un tableau d'entiers
 - Retour : Un tableau d'entiers inversé
 - Exemple de traitement : [\[1, 2, 3\]](#) => [\[3, 2, 1\]](#)
- Exercices supplémentaires (optionnels) :
 - **plus_grande_valeur** : Retourne la plus grande valeur d'un tableau d'entier
 - Paramètre en entrée : Un tableau d'entier
 - Retour : Un entier
 - **PlusPetiteValeur** : Retourne la plus petite valeur d'un tableau d'entier
 - Paramètre en entrée : Un tableau d'entier
 - Retour : Un entier
 - **remplacer_nombre** : Retourne un tableau dont le nombre_1 est remplacée par le nombre_2
 - Paramètres en entrées : Un tableau d'entier, un nombre entier (1^{er}), un nombre entier (2^e)
 - Retour : Un tableau d'entiers (avec nombres remplacés)
 - Test possible : Remplacer le nombre « [3](#) » par le nombre « [8](#) » dans « [mon_tableau](#) »

13. Activités procédurale 2

13.1. Variables et données mutables

Question 1. Dans un code Python, quelle est la différence entre une variable locale et une variable globale ?

Question 2. Soit l'extrait de code Python suivant :

```
#Favoriser l'utilisation de lettres majuscules pour déclarer  
#des constantes en Python (bonne pratique)  
PI = 3.14159                                # Ligne 1  
  
theta = PI / 4                               # Ligne 2  
print("pi / 4:", theta)                     # Ligne 3  
  
theta = theta + PI / 2                       # Ligne 4  
print("theta + pi / 2:", theta)             # Ligne 5
```

- a) Quelles sont les variables déclarées dans ce programme ?
- b) Quel est le rôle du « / » selon les lignes suivantes :
- Lignes 2 et 4 :
 - Lignes 3 et 5 :
- c) Quelle est la signification de `print (...)` ?
- d) Prédisez le comportement de ce programme.
- Quelle est la valeur attribuée aux différentes variables pour chaque ligne de code ?
 - Qu'est-ce qui sera affiché à l'écran ?
- e) Quelle est la différence entre l'opérateur « = » de la ligne 2 versus la ligne 4 du code :

13.2. Opérations arithmétiques et booléennes

Question 1. Prédisez le résultat de ces expressions :

a) $25 - 3 * 4$

b) $(25 - 3) * 4$

c) $100 \% 6$

d) $3 ** 4$

e) $(5 + 3) * (10 - 8)$

f) $100 // 6$

Question 2. Écrivez les expressions pour convertir 100 degrés Fahrenheit en Celsius où chaque valeur est stockée dans sa propre variable.

Question 3. Remplissez les tables logiques (Vrai / Faux) suivantes :

Conditions		ET (AND)
Vrai	Vrai	
Vrai	Faux	
Faux	Vrai	
Faux	Faux	

Conditions		OU (OR)
Vrai	Vrai	
Vrai	Faux	
Faux	Vrai	
Faux	Faux	

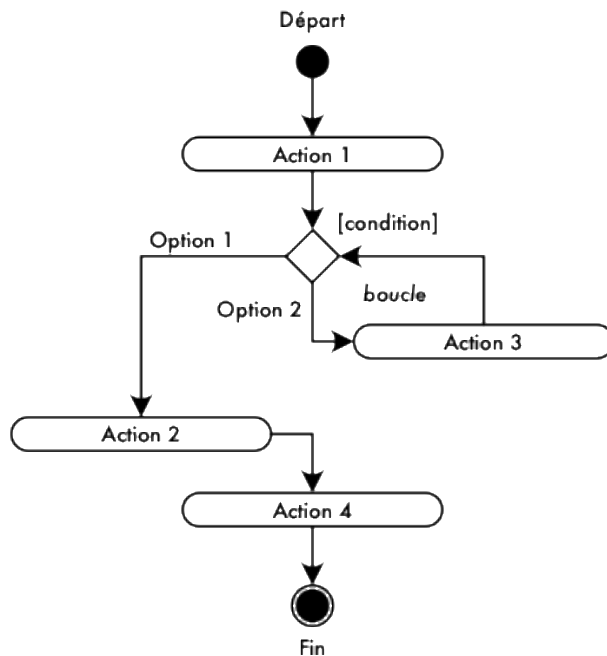
Conditions	NON (NOT)
Vrai	
Faux	

Question 4. Prédisez le résultat (vrai ou faux) des expressions booléennes suivantes :

- a) $(25 - 3) > 4$
- b) `(True and True) or False`
- c) `True and (True or False)`
- d) `True and not False`
- e) `not (2 < 4)`
- f) $(5 \% 2) > 0$

13.3. Diagramme d'activité

Pour décrire des programmes, on peut utiliser des **diagrammes d'activités**. Ils permettent de décrire visuellement, du début à la fin, les étapes que suivra l'ensemble ou une partie d'un programme. Voici le sous-ensemble que nous utiliserons dans le cadre de ce cours :



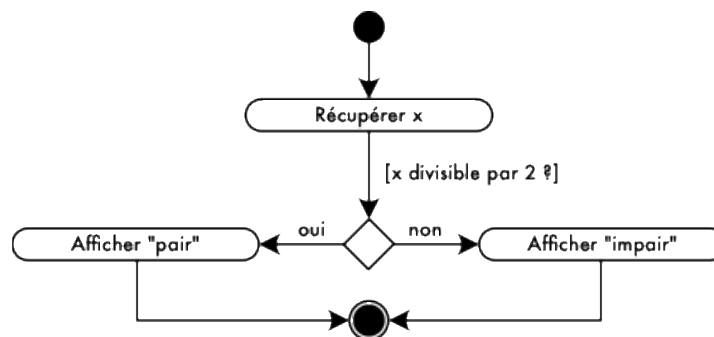
Le cercle plein représente le point de départ du programme, le cercle encerclé la fin.

Les rectangles arrondis représentent une action, comme affecter une variable, récupérer une information de l'utilisateur, écrire dans un fichier ...

Le diamant (ou losange) représente un branchement à la suite d'une condition : on suit le trajet correspondant à la réponse à la condition évaluée entre les crochets.

Le diagramme peut être cyclique : une action ou la suite d'une condition peut mener à une étape déjà passée.

Voici par exemple un court programme déterminant si un nombre est pair ou impair :



Prendre note:

Que le diagramme d'activité devrait être indépendant du langage de programmation et pouvoir être compris par une personne qui ne sait pas programmer. Pour les notions de base, il n'y a pas de problème ($i := 0$, $i := i + 1$, etc.), mais pour les actions dont la syntaxe peut varier d'un langage de programmation à un autre, favoriser l'usage d'une description textuelle de l'action, tel que : « Lire l'élément à la position "i" ».

Il est également permis de définir préalablement une variable, par exemple « nbElements := taille du tableau » (dans une boîte d'action). Puis de réutiliser nbElements dans une opération mathématique.

Question 1. Reprenons le programme de conversion de Fahrenheit à Celsius. Imaginons qu'il s'agit de la température d'un volume d'eau.

- a) Faites le diagramme d'activité pour un programme qui afficherait, selon la température en Celsius, si l'eau gèle (moins de 0 degrés Celsius) ou bout (plus de 100 degrés Celsius).
- b) Écrivez maintenant la ou les conditions " if " pour reproduire ce programme.

Question 2. Vous devez écrire un programme qui doit :

- Récupérer deux nombres, x et y
- Calculer le résultat de x / y (diviseur et reste), **seulement en utilisant des additions et des soustractions** et en assumant que y n'est pas égal à 0.

Vous pouvez considérer que **x sera toujours plus grand que y** et que les **deux nombres** seront toujours **positifs**. Vous pouvez également affecter des variables additionnelles.

Un indice : Souvenez-vous comment vous avez appris à diviser des nombres.

- Est-ce que des étapes se répétaient ?
- À quel moment on arrête cette répétition ?
- Quelle est la condition à vérifier ?

Une fois le diagramme d'activité créé, écrivez le programme équivalent en Python, seulement sur papier pour l'instant. Pour la récupération des valeurs x et y, vous pouvez tout simplement initialiser deux variables avec les valeurs de votre choix.

14. Activité en laboratoire 2

Les objectifs de ce laboratoire sont :

- Se familiariser avec un environnement de développement intégré, ici Visual Studio Code, et le débogueur pour Python ;
- Être capable de suivre le déroulement d'un programme et en observer les effets ;
- Identifier et corriger des problèmes ;
- Compléter des fonctionnalités ;
- Écrire des routines de test pour valider le bon fonctionnement d'un sous-ensemble du programme ;
- Documenter adéquatement un programme.

Votre tuteur ou tutrice vous guidera dans les prochaines étapes.

Ouvrez d'abord le dossier des fichiers de la problématique dans Visual Studio Code. Dans les fichiers, vous trouverez "[labo.py](#)". L'ensemble du travail sera dans ce fichier. Le but de ce programme est de récupérer une liste de nombres de l'utilisateur, puis afficher quelle était la valeur maximale.

Tâche 1 – Analyse et complétion du programme

En parcourant le code, vous remarquerez deux commentaires "**À FAIRE**", ils représentent deux tâches à compléter :

- Un pour la récupération des valeurs dans une liste ;
- L'autre pour l'affichage des résultats.

Avant de vous lancer dans la complétion du code. Exécutez d'abord le programme dans le débogueur. Ajoutez des points d'arrêts (*breakpoints*) aux endroits où il y a des conditions, et interagissez avec le programme. Qu'est-ce que vous remarquez ? Par quel chemin l'exécution passe ? Est-ce toujours linéaire ? Pensez aussi à observer l'état des variables en cours d'exécution.

Tâche 2 – Tests automatisés

Dans le code du laboratoire, vous trouverez également une fonction "`test_trouver_maximum()`". L'objectif de cette fonction est de tester automatiquement le bon fonctionnement de la méthode chargée de trouver la valeur maximale. Elle est cependant pratiquement vide. Complétez-la en ajoutant quelques (3 ou 4) tests permettant de valider si la méthode fonctionne bien. Par exemple, vous pourriez ajouter la ligne :

```
assert(trouve_maximum([1, 5, 3]) == 5)
```

pour vérifier si la fonction trouve bel et bien 5. Si la condition est fausse, une erreur sera générée et pourra être récupérée par le débogueur. Pour exécuter la fonction de test, modifiez la ligne commentée en début de fonction principale. Ceci aura pour effet de lancer les tests avant d'interagir avec l'utilisateur.

Tâche 3 – Identification et correction d'un problème

Vous remarquerez normalement un problème dans l'implémentation de la fonction qui se charge de trouver la valeur maximale. **Identifiez le problème avec le débogueur et corrigez la fonction.**

Tâche 4 – Documentation d'une fonction

Dans cette même fonction de recherche de valeur maximale, vous avez peut-être remarqué que les commentaires ne sont pas particulièrement pertinents. Après avoir consulté la présentation sur la documentation du code, proposez des changements aux commentaires pour mieux guider les futurs lecteurs du code.

**Une fois les tâches complétées, vous pouvez débiter (ou continuer) la résolution de la problématique.*

15. Activités procédurale 3

15.1. Types et manipulation de listes

Question 1. Quel est le type des variables déclarées dans chacun de ces énoncés ?

```
a = 12
b = 12.0
c = "12.0"
d = [12]
```

Question 2. Prenons cet extrait de code :

```
a = 2*"12"
print(a)
```

- Prédisez le résultat à l'écran.

**Le tuteur ou la tutrice va maintenant présenter le résultat à l'écran.*

- Était-ce le résultat attendu ?
- Comment peut-on arriver au résultat que vous vouliez ?
- Êtes-vous maintenant en mesure de prédire le résultat de ce code ?

```
a = 2*[12]
print(a)
```

Question 3. Que fait ce code ?

```
fruits = ["pomme", "banane", "orange"]
fruits.append("kiwi")
fruits[1] = "fraise"
dernier = fruits.pop()
print(fruits)
print(f"Enlevé: {dernier}")
```

Question 4. Pour une chaîne de texte d'au moins un caractère, comment fait-on pour ...

- a) Récupérer le premier caractère de la chaîne ?
- b) Le dernier ?
- c) Les trois premiers ?
- d) Afficher chaque caractère sur ligne séparée ?

15.2. Entrées et sorties**Question 1.** Que fait ce programme ?

```
reponse = input("Continuez? (o/n): ")
while reponse.lower() != 'n':
    print("On continue...")
    reponse = input("Continuez? (o/n): ")
print("Au revoir!")
```

Question 2. Que fait ce programme ?

```
somme = 0
fichier = open("nombres.txt", "r")
for ligne in fichier:
    somme += int(ligne)
print("Somme:", somme)
```

Question 3. Que fait ce code ?

```
fichier = open("sortie.txt", "w")
for i in range(1, 11):
    fichier.write(f"{i} au carré = {i**2}\n")
```

15.3. Définition de fonctions

Question 1. Transformez ce code pour que le calcul d'aire d'un carré puisse être réutilisé comme fonction.

```
longueur = int(input("Longueur ?"))
aire = longueur * longueur
print("Aire:", aire)
```

Question 2. Prédisez le comportement de ce programme :

```
a = 10

def func():
    a = 15
    print(a)

print(a)
func()
print(a)
```

Le tuteur ou la tutrice va maintenant présenter le résultat à l'écran.

Était-ce le résultat attendu ?

Question 3. Lorsqu'une fonction appelle une autre fonction (dans un même module), est-ce que l'ordre de définition des fonctions est important dans le module ?

15.4. Définition de module

Question 1. En Python, qu'est-ce qu'un module ?

Question 2. Que fait la ligne suivante dans un module ?

```
if __name__ == "__main__":
```

Question 3. Expliquer ce que fait chacune des instructions d'importation de module suivante :

- **import** mathematique
- **from** mathematique **import** additionner
- **from** mathematique **import** *
- **import** mathematique **as** math
- **from** mathematique **import** additionner **as** sum

16. Validation pratique de la solution à la problématique

L'ordre de présentation des équipes sera aléatoire et sera diffusé sur le site de l'APP. Lors de la validation, vous aurez à démontrer le bon fonctionnement de votre solution et expliquer comment vous avez résolu le problème. Référez-vous à la grille de la section 10.4 (dernières colonnes) pour connaître les critères d'évaluation. La personne évaluant votre travail pourrait vous poser des questions individuellement à chaque membre de l'équipe – assurez-vous que chaque membre soit en mesure de décrire l'ensemble de la solution.

17. Deuxième tutorat

Le deuxième tutorat est l'occasion pour vous de partager vos solutions à la problématique et confirmer vos acquis. Votre tuteur ou tutrice vous guidera dans la discussion. Pensez à prendre des notes, par exemple en dessinant un schéma relationnel des concepts abordés. Pensez à relire la liste des connaissances à acquérir à la section 5 et n'hésitez surtout pas à poser des questions si certains de ces concepts ne sont pas encore clairs pour vous.

18. Comparaison avec le C++

Vous avez certainement remarqué que le langage de programmation dans cette unité d'APP, le Python, est différent de celui utilisé pour votre projet de session sur Arduino, le C++. Tout au long de votre carrière en robotique, vous aurez à travailler avec de langages différents selon votre domaine d'application. Si vous vous retrouvez à écrire du code embarqué bas niveau, ce sera fort probablement en C ou C++ pour les prochaines années, et peut-être en Rust ou Zig si c'est un nouveau projet. Si vous travaillez sur des interfaces utilisateurs, il y a de fortes chances qu'elles soient basées sur des technologies web, et vous aurez à maîtriser des langages comme le Javascript ou le Typescript, sans compter la panoplie d'environnement de développement « backend / frontend » qui se renouvellent continuellement. La bonne nouvelle est que tous les langages de programmation dit procéduraux ont énormément d'éléments en commun comme la portée lexicale, l'évaluation immédiate et les structures de contrôle. C'est justement ce qui est à l'étude dans cette unité d'APP. De plus, les outils modernes d'IA générative comme Claude ou ChatGPT sont particulièrement habiles pour traduire un programme d'un langage à un autre, n'hésitez pas à les utiliser dans votre apprentissage.

Cependant, pour vous aider dans votre compréhension de la programmation dans le contexte de votre projet de session, nous allons exposer quelques similitudes (et différences) importantes entre le C++ et le Python. Reprenons l'exemple de code qui vous a été montré dans le cours de projet :

```

001: /*
002: Projet: Le nom du script
003: Equipe: Votre numero d'equipe
004: Auteurs: Les membres auteurs du script
005: Description: Breve description du script
006: Date: Derniere date de modification
007: */
008:
009: /*
010: Inclure les librairies de fonctions que vous voulez utiliser
011: */
012: #include <LibRobus.h>
013:
014: /*
015: Variables globales et defines
016: -> defines...
017: -> L'ensemble des fonctions y ont acces
018: */
019:
020: bool bumperArr;
021: int vertpin = 48;
022: int rougepin = 49;
023: bool vert = false;
024: bool rouge = false;
025: int etat = 0; // = 0 arrêt 1 = avance 2 = recule 3 = TourneDroit 4 = TourneGauche
026: int etatPast = 0;
027: float vitesse = 0.40;
028:
029: /*
030: Vos propres fonctions sont creees ici
031: */
032:
033: void beep(int count){
034:     for(int i=0;i<count;i++){
035:         AX_BuzzerON();
036:         delay(100);
037:         AX_BuzzerOFF();
038:         delay(100);

```

```

039:  }
040:  delay(400);
041: }
042:
043: void arret(){
044:   MOTOR_SetSpeed(RIGHT, 0);
045:   MOTOR_SetSpeed(LEFT, 0);
046: };
047:
048: void avance(){
049:   MOTOR_SetSpeed(RIGHT,vitesse);
050:   MOTOR_SetSpeed(LEFT, vitesse);
051: };
052:
053: void recule(){
054:   MOTOR_SetSpeed(RIGHT, -0.5*vitesse);
055:   MOTOR_SetSpeed(LEFT, -vitesse);
056: };
057:
058: void tourneDroit(){
059:   MOTOR_SetSpeed(RIGHT, 0.5*vitesse);
060:   MOTOR_SetSpeed(LEFT, -0.5*vitesse);
061: };
062:
063: void tourneGauche(){
064:   MOTOR_SetSpeed(RIGHT, -0.5*vitesse);
065:   MOTOR_SetSpeed(LEFT, 0.5*vitesse);
066: };
067:
068: /*
069: Fonctions d'initialisation (setup)
070: -> Se fait appeler au debut du programme
071: -> Se fait appeler seulement un fois
072: -> Generalement on y initilise les varibbles globales
073: */
074: void setup(){
075:   BoardInit();
076:
077:   //initialisation
078:   pinMode(vertpin, INPUT);
079:   pinMode(rougepin, INPUT);
080:   delay(100);
081:   beep(3);
082: }
083:
084: /*
085: Fonctions de boucle infini
086: -> Se fait appeler perpetuellement suite au "setup"
087: */
088: void loop() {
089:   etatPast = etat;
090:   bumperArr = ROBUS_IsBumper(3);
091:   if (bumperArr){
092:     if (etat == 0){
093:       beep(2);
094:       etat = 1;
095:     }
096:     else{
097:       beep(1);
098:       etat = 0;
099:     }
100:   }
101:
102:   vert = digitalRead(vertpin);
103:   rouge = digitalRead(rougepin);
104:   if (etat > 0){
105:     if (vert && rouge){ // aucun obstacle => avance
106:       etat = 1;
107:     }
108:     if (!vert && !rouge){ // obstacle devant => recule
109:       etat = 2;

```

```
110:     }
111:     if (!vert && rouge){ // obstacle à gauche => tourne droit
112:         etat = 3;
113:     }
114:     if (vert && !rouge){ // obstacle à droite => tourne gauche
115:         etat = 4;
116:     }
117: }
118:
119: if (etatPast != etat){
120:     arret();
121:     delay(50);
122: }
123: else{
124:     switch (etat)
125:     {
126:     case 0:
127:         arret();
128:         break;
129:     case 1:
130:         avance();
131:         break;
132:     case 2:
133:         recule();
134:         break;
135:     case 3:
136:         tourneDroit();
137:         break;
138:     case 4:
139:         tourneGauche();
140:         break;
141:     default:
142:         avance();
143:         etat = 1;
144:         break;
145:     }
146: }
147: delay(200);
148: }
```

Si on traduisait directement le code en Python, on pourrait obtenir quelque chose comme ceci. Attention : ne tentez pas de copier ce code dans un fichier et l'exécuter sur votre ordinateur : le code de support de libRobus n'existe pas en Python et, contrairement à l'environnement Arduino, les fonctions loop() et setup() n'ont pas de sens particulier.

```

001: """
002: Projet: Le nom du script
003: Equipe: Votre numero d'equipe
004: Auteurs: Les membres auteurs du script
005: Description: Breve description du script
006: Date: Derniere date de modification
007: """
008:
009: # Inclure les librairies de fonctions que vous voulez utiliser
010: from librobust import *
011:
012: # Variables globales et defines
013: # -> defines...
014: # -> L'ensemble des fonctions y ont acces
015:
016: bumper_arr = False
017: vert_pin = 48
018: rouge_pin = 49
019: vert = False
020: rouge = False
021: etat = 0 # = 0 arrêt 1 = avance 2 = recule 3 = TourneDroit 4 = TourneGauche
022: etat_past = 0
023: vitesse = 0.40
024:
025: # Vos propres fonctions sont creees ici
026:
027: def beep(count):
028:     for i in range(count):
029:         AX_BuzzerON()
030:         delay(100)
031:         AX_BuzzerOFF()
032:         delay(100)
033:     delay(400)
034:
035: def arret():
036:     MOTOR_SetSpeed(RIGHT, 0)
037:     MOTOR_SetSpeed(LEFT, 0)
038:
039: def avance():
040:     MOTOR_SetSpeed(RIGHT, vitesse)
041:     MOTOR_SetSpeed(LEFT, vitesse)
042:
043: def recule():
044:     MOTOR_SetSpeed(RIGHT, -0.5 * vitesse)
045:     MOTOR_SetSpeed(LEFT, -vitesse)
046:
047: def tourne_droit():
048:     MOTOR_SetSpeed(RIGHT, 0.5 * vitesse)
049:     MOTOR_SetSpeed(LEFT, -0.5 * vitesse)
050:
051: def tourne_gauche():
052:     MOTOR_SetSpeed(RIGHT, -0.5 * vitesse)
053:     MOTOR_SetSpeed(LEFT, 0.5 * vitesse)
054:
055: # Fonctions d'initialisation (setup)
056: # -> Se fait appeler au debut du programme
057: # -> Se fait appeler seulement un fois
058: # -> Generalement on y initilise les varibbles globales
059: def setup():
060:     BoardInit()
061:
062:     # initialisation
063:     pinMode(vert_pin, INPUT)

```

```

064:     pinMode(rouge_pin, INPUT)
065:     delay(100)
066:     beep(3)
067:
068: # Fonctions de boucle infini
069: # -> Se fait appeler perpetuellement suite au "setup"
070: def loop():
071:     global etat, etat_past, bumper_arr, vert, rouge
072:
073:     etat_past = etat
074:     bumper_arr = ROBUS_IsBumper(3)
075:
076:     if bumper_arr:
077:         if etat == 0:
078:             beep(2)
079:             etat = 1
080:         else:
081:             beep(1)
082:             etat = 0
083:
084:     vert = digitalRead(vert_pin)
085:     rouge = digitalRead(rouge_pin)
086:
087:     if etat > 0:
088:         if vert and rouge: # aucun obstacle => avance
089:             etat = 1
090:         if not vert and not rouge: # obstacle devant => recule
091:             etat = 2
092:         if not vert and rouge: # obstacle à gauche => tourne droit
093:             etat = 3
094:         if vert and not rouge: # obstacle à droite => tourne gauche
095:             etat = 4
096:
097:     if etat_past != etat:
098:         arret()
099:         delay(50)
100:     else:
101:         # Switch-case converted to if-elif-else
102:         if etat == 0:
103:             arret()
104:         elif etat == 1:
105:             avance()
106:         elif etat == 2:
107:             recule()
108:         elif etat == 3:
109:             tourne_droit()
110:         elif etat == 4:
111:             tourne_gauche()
112:         else: # default case
113:             avance()
114:             etat = 1
115:
116:     delay(200)

```

La structure générale est très similaire : on importe des fonctions de « libRobus », on définit des variables globales et des fonctions utiles, et on reconnaît différentes structures de contrôle comme le « if ». Analysons plus en détails les différences principales.

18.1. Importation

En C++, on utilise cette ligne pour importer la librairie libRobus :

```
012: #include <LibRobus.h>
```

Celle-ci a littéralement pour effet d'insérer la totalité du contenu du fichier LibRobus.h à cet endroit. Donc, si une fonction est déclarée, elle devient automatiquement accessible par son nom, comme pour les fonctions déjà en place comme `arret()` ou `recule()`. En Python, c'est légèrement différent : importer un module avec « `import librobust` » a pour effet de rendre accessibles les éléments déclarés dans le module librobust. Pour y accéder, il faudrait préfixer chaque appel par le nom du module, par exemple en faisant « `librobust.MOTOR_SetSpeed(LEFT, vitesse)` ». Or, nous pouvons utiliser la construction « `from <module> import <term1, term2, ...>` » qui permet d'intégrer directement à la portée lexicale en cours un ou plusieurs éléments du module. Avec « `*` », on importe l'ensemble, comme en C++. Ainsi, la ligne

```
010: from librobust import *
```

importe l'ensemble des éléments du module librobust, et toutes les fonctions qui y sont définies deviennent accessibles. La conversion en minuscule de « librobust » est simplement pour respecter les lignes directrices de style en Python. Techniquement, rien n'empêche de nommer un module « libRobust » et son fichier associé « libRobust.py », mais c'est déconseillé pour des questions de portabilité du code d'un système à l'autre¹.

18.2. Déclaration de variables, énoncés et commentaires

Comme le code en C++, le code en Python déclare certaines variables globales. Regardons d'abord le code original :

```
020: bool bumperArr;  
021: int vertpin = 48;  
022: int rougepin = 49;  
023: bool vert = false;  
024: bool rouge = false;  
025: int etat = 0; // = 0 arrêt 1 = avance 2 = recule 3 = TourneDroit 4 =  
TourneGauche  
026: int etatPast = 0;  
027: float vitesse = 0.40;
```

¹ Certains systèmes, comme Windows, ne font pas la distinction entre « libRobust.py » et « librobust.py », mais sous Linux et macOS ces deux fichiers peuvent co-exister dans le même dossier. C'est donc pour éviter la confusion qu'on recommande fortement de n'utiliser que des minuscules pour les noms de fichiers.

Et maintenant l'équivalent Python :

```
016: bumper_arr = False
017: vert_pin = 48
018: rouge_pin = 49
019: vert = False
020: rouge = False
021: etat = 0    # = 0 arrêt 1 = avance 2 = recule 3 = TourneDroit 4 =
    TourneGauche
022: etat_past = 0
023: vitesse = 0.40
```

Une première différence saute aux yeux : les mots-clés `bool`, `int` et `float`. En effet, si en Python l'affectation à une variable définit en même temps son type (ex. « `a = 12.2` » fait en sorte que la variable `a` est de type `float` et aura pour valeur 12.2), en C++ et pour plusieurs autres langages dits « typés statiquement » on doit explicitement déclarer le type de chaque variable. Ainsi, la variable `bumper_arr` sera obligatoirement un `float`, et tenter d'affecter une valeur d'un autre type provoquera une erreur s'il n'y a pas de conversion implicite possible. Par exemple, il serait possible d'affecter « 5 » à `vitesse` puisque que l'entier « 5 » sera automatiquement converti en « 5.0 », mais tenter d'affecter une chaîne de caractères comme « `abcd` » provoquerait une erreur de compilation. Nous reviendrons sur ceci dans un prochain cours de votre parcours. L'autre petite différence est l'utilisation de `True` et `False` avec des premières lettres en majuscule. C'est un simple choix de style du Python : les valeurs constantes (éléments qu'on ne peut pas changer en cours d'exécution) débutent par des consonnes pour les distinguer des variables mutables.

La deuxième grande différence est dans l'absence du point-virgule en Python. En C++ et plusieurs autres langages inspirés du C (C#, Objective C, Java, ...), le point-virgule sert à terminer les énoncés. Les sauts de ligne (le « Enter » qu'on tape pour passer à la prochaine ligne) ont la même signification qu'un espace, c'est-à-dire un simple séparateur entre éléments d'un même énoncé. Ainsi, il est tout à fait possible en C++ d'écrire quelque chose comme :

```
int a =
    15
;
```

ou

```
MOTOR_SetSpeed(RIGHT,
    -0.5 * vitesse
);
```

Finalement, la déclaration des commentaires est légèrement différente : le « `#` » a la même signification que le « `//` », c'est-à-dire que tous les caractères suivant ce symbole sont ignorés pour cette ligne. Les commentaires sur plusieurs lignes (« `/* ... */` » en C++) n'existent pas réellement en Python. On doit plutôt répéter le symbole « `#` », comme aux lignes 12 à 14.

18.3. Blocs et portée lexicale

Nous avons vu comment on peut regrouper des instructions en blocs (ce que le livre de référence nomme les « blocs imbriqués ») et quel effet cela peut avoir sur la portée lexicale des variables. Le même principe est bien présent en C++, mais se présente différemment. En Python, les blocs sont définis par leur niveau de retrait. Ainsi, dans une définition de fonction comme celle-ci, les lignes 52 et 53 appartiennent au bloc et à la portée lexicale de la fonction `tourne_gauche()` :

```
051: def tourne_gauche():
052:     MOTOR_SetSpeed(RIGHT, -0.5 * vitesse)
053:     MOTOR_SetSpeed(LEFT, 0.5 * vitesse)
```

L'équivalent en C++ ressemble plutôt à ceci :

```
063: void tourneGauche(){
064:     MOTOR_SetSpeed(RIGHT, -0.5*vitesse);
065:     MOTOR_SetSpeed(LEFT, 0.5*vitesse);
066: };
```

On reconnaît les points-virgules pour terminer les énoncés, mais deux nouveaux symboles s'ajoutent : les accolades (« { ... } »). Elles servent à délimiter précisément la portée lexicale d'un bloc. En effet, comme le nombre d'espaces ou même de sauts de ligne n'a aucune signification en C++, le fait d'aligner en retrait des lignes de code ne les regroupent pas ensemble comme en Python. Il faut donc délimiter le début et la fin d'un bloc par les accolades. C'est particulièrement important avec des blocs de contrôle comme le « if » ici :

```
092:     if (etat == 0){
093:         beep(2);
094:         etat = 1;
095:     }
```

Si on omet les accolades, le code est non seulement valide, mais la ligne 94 qui affecte « 1 » à la variable `etat` sera toujours évaluée, même si la variable `etat` n'est pas égale à zéro ! C'est pourquoi il est fortement recommandé de toujours mettre des accolades en C++, même si le bloc attaché à une condition ne fait qu'une ligne. Il est aussi recommandé d'aligner les blocs en retrait, comme en Python. Même si le retrait n'a pas de signification particulière, il facilite grandement l'interprétation du code. On remarque aussi l'utilisation des deux points « : » à la suite de la condition du *if* et du *else*. Sauf l'utilisation de « *elif* » au lieu de « *else if* » pour insérer une nouvelle condition est un peu différente, mais la structure demeure la même.

Finalement, vous remarquerez l'utilisation du mot-clé « *global* » à la ligne 71 pour assurer que les affectations aux variables globales comme `etat` ne seront pas traitées comme une nouvelle variable locale masquant l'originale. En C++, les variables définies globalement sont automatiquement accessibles autant en lecture qu'en écriture, et il aurait fallu déclarer une nouvelle variable du même nom, ce qui aurait d'ailleurs le même effet de masquage. L'équivalent d'un mot-clé « *global* » en C++ n'est donc pas nécessaire dans cette situation.

18.4. Déclaration de fonctions

Dans les définitions de fonctions, on voit deux choses qui peuvent sembler être un élément de style, mais qui ont des répercussions plus subtiles : l'utilisation de « void » en C++ plutôt que « def » et l'accolade ouverte « { » plutôt que les deux-points « : ». Nous avons vu que les accolades permettaient de définir la portée lexicale d'un bloc de code en C++. Contrairement aux blocs de contrôle comme le « if », elles sont obligatoires pour le corps des fonctions. L'accolade ouverte ne fait donc que commencer la portée lexicale de la fonction, et aurait pu être seule sur une ligne séparée. C'est d'ailleurs un style populaire en C++ et d'autres langages similaires pour faciliter l'identification à l'œil nu de blocs de code. Voici de quoi pourrait avoir l'air la fonction `tourneGauche` suivant ce style :

```
void tourneGauche()
{
    MOTOR_SetSpeed(RIGHT, -0.5*vitesse);
    MOTOR_SetSpeed(LEFT, 0.5*vitesse);
};
```

Le rôle de « void » est plus particulier. Si « def » en Python sert simplement à annoncer la définition d'une fonction, en C++ les parenthèses sont suffisantes. Or, comme pour les variables, il est impératif de déclarer le type de retour d'une fonction. Ici, toutes les fonctions ont un retour « void », c'est-à-dire qu'elle ne retourne aucune valeur à la fin de leur exécution. Ça ne veut pas dire qu'elles n'ont pas d'effet. On peut généralement supposer qu'une fonction sans retour modifiera l'état global du programme (donc les valeurs de ses variables globales), ou interagira avec l'extérieur (en écrivant dans un fichier, par exemple). Ici, les fonctions ont un clairement un effet : ajuster les vitesses des moteurs. Les vitesses ne sont pas nécessairement des variables globales directement accessibles par notre programme, mais appeler ces fonctions auront un effet direct sur le système dans son entier. Pour comprendre le rôle du type de retour en C++, voici un exemple de fonction calculant le carré d'un nombre entier. En Python, on aurait :

```
def square(v):
    return v*v
```

Alors qu'en C++, ce serait :

```
int square(int v) {
    return v*v;
}
```

Contrairement à la version en Python, la fonction en C++ ne fonctionnera qu'avec des entiers². C'est une contrainte des langages statiquement typés : une fonction ne peut pas retourner plus d'un type de valeur. C'est un autre concept qui sera abordé plus en détails dans un prochain cours de votre parcours.

² Qu'arrive-t-il si on appelle la fonction C++ avec un float (ex. `float b = square(3.5);`) ? La valeur 3.5 est d'abord implicitement convertie au plus petit entier, 3, et b aura donc la valeur 9 plutôt que 12.25. Un compilateur relativement moderne produira un avertissement à la conversion de la valeur 3.5, mais ça demeure un programme valide. I

18.5. Opérateurs booléens et *switch-case*

En Python, on construit nos conditions avec un ensemble de `and`, `or`, `not` et signes de comparaisons comme le « `>` ». Le C++ reprend les mêmes concepts, mais utilise des symboles pour `and`, `or` et `not` : `&&`, `||` et `!`. L'égalité utilise aussi le double « `=` » pour distinguer la comparaison de l'affectation. Attention : en C++, il existe aussi les opérateurs binaires `&`, `|` et `~`. Ils ont des comportements différents, même si le résultat dans une condition *if* peut sembler être le même. Ces différences sont liées au système de typage et la manipulation de nombres binaires et elles ne seront pas abordées ici.

Finalement, la construction « *switch-case* » n'existait pas en Python avant la version 3.10 sortie en 2021. C'est pour cela qu'une construction de type *if* et *elif* en séquence est utilisé ici, car elle est encore très présente dans le code Python encore disponible aujourd'hui. Elle a aussi l'avantage d'être plus robuste : l'absence d'un « `break` » fait en sorte que l'exécution peut « sauter » d'un bloc à l'autre.³ Ce n'est pas le cas avec une séquence de conditions. Si on veut transformer le « *switch-case* » en un « *match-case* » en Python 3.10+, le bloc pourrait ressembler à ceci :

```
match etat:
    case 0:
        arret()
    case 1:
        avance()
    case 2:
        recule()
    case 3:
        tourne_droit()
    case 4:
        tourne_gauche()
    case _: # cas par défaut, équivalent de default en C++.
        avance()
        etat = 1
```

³ Vous avez peut-être remarqué que le « *switch-case* » ne ressemble pas aux autres structures de contrôle comme le *if* ou le *while*. Par exemple, où sont les accolades délimitant les blocs pour chaque cas ? C'est un artefact surtout historique qui trahit les origines très proches du matériel du langage C.

